



ค่ายโอลิมปิกวิชาการ วิชาคอมพิวเตอร์

โครงสร้างข้อมูลแบบเชิงเส้น (Linear data structures)

อ.ลือพล พิพานเมฆาภรณ์

luepol.p@sci.kmutnb.ac.th

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ



หัวข้อ

- โครงสร้างข้อมูลเชิงเส้น (Linear data structure)
- อาร์เรย์ (array)
- สแตก (Stack)
- คิว (Queue)
 - คิวแบบง่าย (simple queue)
 - คิววงกลม (circular queue)
 - คิวลำดับความสำคัญ (priority queue)



โครงสร้างข้อมูลเชิงเส้น (Linear data structure)

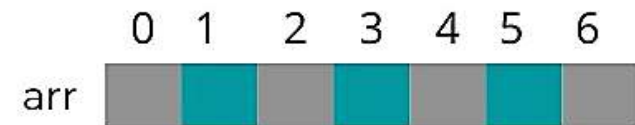
- **โครงสร้างข้อมูล (Data Structure)** เป็นเครื่องมือในการจัดเก็บข้อมูลในรูปแบบที่มีโครงสร้างไว้ในคอมพิวเตอร์ เพื่อช่วยให้สามารถประมวลผลข้อมูลได้อย่างมีประสิทธิภาพ แบ่งออกเป็น 2 ประเภท ได้แก่
 - **โครงสร้างข้อมูลแบบเชิงเส้น (Linear data structure)** เป็นโครงสร้างข้อมูลที่จัดเก็บข้อมูลในลักษณะต่อเนื่องกัน เช่น อาร์เรย์ สแตก และคิว เป็นต้น
 - **โครงสร้างข้อมูลแบบไม่เชิงเส้น (Non-linear data structure)** โครงสร้างที่ไม่มีคุณสมบัติของเชิงเส้น สามารถใช้แสดงความสัมพันธ์ของข้อมูลที่ซับซ้อนได้มากกว่าโครงสร้างข้อมูลแบบเชิงเส้น เช่น ตรี (tree) กราฟ (graph)

อาร์เรย์ (Array)

- อาร์เรย์ (array) รายการข้อมูลที่จัดเก็บไว้ในหน่วยความจำ โดยที่ข้อมูลแต่ละตัวจะถูกเก็บไว้ในตำแหน่งที่ติดกัน ซึ่งจะสามารถเข้าถึงข้อมูลแต่ละตัวได้โดยระบุเลขดัชนี (index)
 - ใน C หรือ C++ เลขดัชนีจะเริ่มต้นที่ตำแหน่ง 0 เสมอ (สมาชิกตัวแรก)
- ในการใช้งานอาร์เรย์จะต้องมีการกำหนดขนาด (size) ของอาร์เรย์ที่แน่นอน

100	200	300	400
arr[0]	arr[1]	arr[2]	arr[3]

อาร์เรย์ (Array)



arr → Array Variable
[0] → Index of the element to be accessed



การเตรียมใช้งานอาร์เรย์

- ระบุประเภทข้อมูลในอาร์เรย์ (array type) และขนาดของอาร์เรย์ (array size)
 - `int arr[10];` `// arr[0], arr[1], ..... , arr[9]`
- ระบุประเภทพร้อมกำหนดค่าเริ่มต้น
 - `int arr[] = {10, 20, 30, 50, 100, -100};` `// arr[0]=10, arr[1]=20,...,arr[5] = -100`



Dynamic memory allocation in array

- โดยทั่วไปการกำหนดขนาดของอาร์เรย์จะมีลักษณะเป็น **static** กล่าวคือจะต้องกำหนดขนาดไว้ล่วงหน้า และไม่สามารถเปลี่ยนแปลงขนาดของอาร์เรย์ได้ขณะรันโปรแกรม
- อย่างไรก็ตาม ในภาษาซีมีวิธีการกำหนดขนาดของอาร์เรย์ในขณะรันโปรแกรมได้ โดยใช้ฟังก์ชัน `malloc()`

```
array = (type *)malloc(N*sizeof(type));
```

ตัวอย่างการใช้งานใน C

```
1  #include<stdio.h>
2
3  int main() {
4      int *ptr,i,n;
5      printf("Enter number of items : ");
6      scanf("%d",&n);
7
8      ptr = (int*)malloc(n * sizeof(int));
9      for(i=0; i<n;i++)
10         { ptr[i] = i;
11         }
12
13     for(i=0; i<n;i++)
14         { printf("%d ", ptr[i]);
15         }
16
17
18     return 0;
19 }
```

```
Enter number of items : 10
0 1 2 3 4 5 6 7 8 9
```



ตัวอย่างการใช้งานใน C++

```
#include <iostream>

using namespace std;

int main() {
    int *ptr, i , n;

    cout<<"Enter number of items: ";
    cin>>n;

    ptr = new int[n];

    for(i = 0; i<n; i++)
        ptr[i] = i;

    for(i = 0; i<n; i++)
        cout<< ptr[i]<< " ";

    return 0;
}
```

```
Enter number of items : 10
0 1 2 3 4 5 6 7 8 9
```



ปัญหาทั่วไปในการทำงานใช้อาร์เรย์

- การค้นหาข้อมูล (Searching) ในอาร์เรย์ขนาด n จำนวน
 - กำหนดให้ k เป็นอินพุตที่ต้องการค้นหา
 - ค้นหาตำแหน่งสมาชิกในอาร์เรย์ที่มีค่าเท่ากับ k
 - อัลกอริทึมค้นหาข้อมูลแบบง่าย ได้แก่ linear search (sequential search)
- การเรียงลำดับข้อมูล (Sorting) ในอาร์เรย์ขนาด n จำนวน
 - ระบุรูปแบบในการเรียงข้อมูล: น้อยไปมาก (ascending order) มากไปน้อย (descending order)
 - เรียงข้อมูลในอาร์เรย์ตามรูปแบบที่กำหนด
 - อัลกอริทึมเรียงข้อมูลแบบง่าย ได้แก่ bubble sort

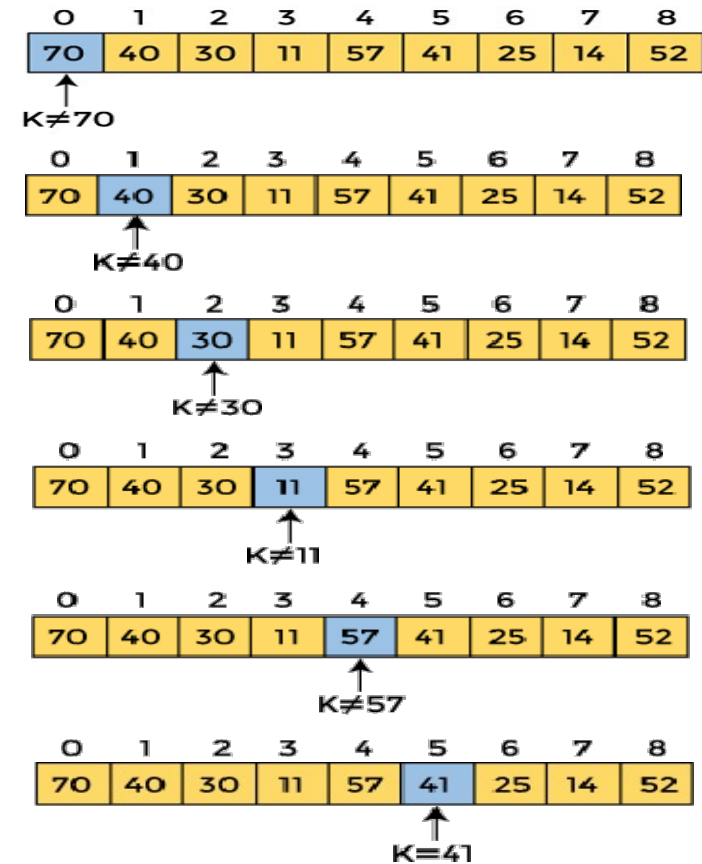
Linear search algorithm

กำหนดให้อาร์เรย์มีข้อมูล 9 ตัว

0	1	2	3	4	5	6	7	8
70	40	30	11	57	41	25	14	52

ต้องการค้นหาสมาชิกในอาร์เรย์ที่มีค่าเท่ากับ 41

$k = 41$





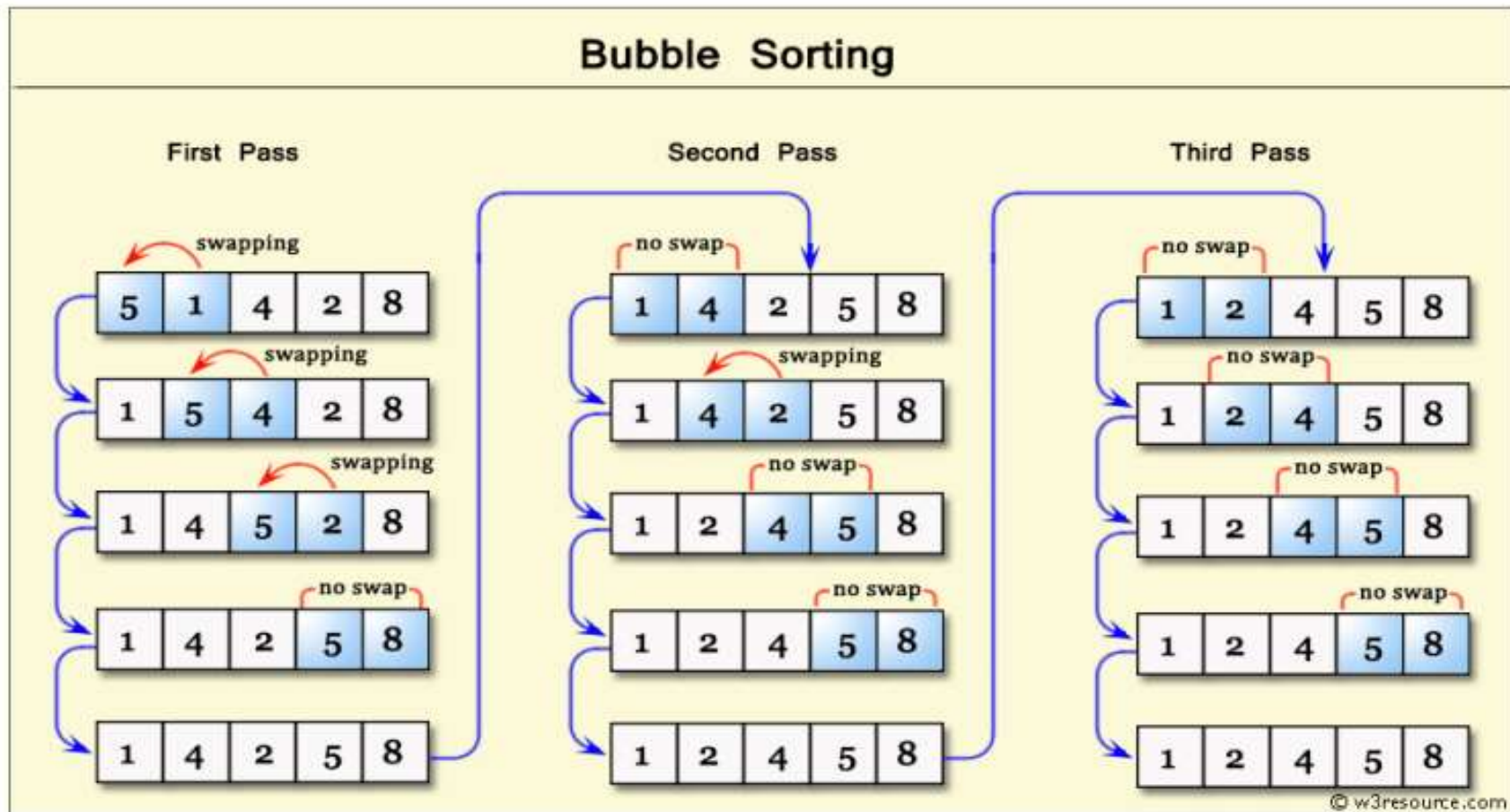
Linear search algorithm

```
1  #include<stdio.h>
2
3  int main() {
4      int arr[] = {70,40,30,11,57,41,25,14,52};
5      int n = sizeof(arr)/sizeof(int);
6      int k = 41, i=0;
7
8      while( (arr[i] != k) && (i < n))
9          i++;
10
11      if(i<n)
12          printf("found item %d at index %d in arr", k, i);
13      else
14          printf("no %d in arr", k);
15
16      return 0;
17 }
```

found item 41 at index 5 in arr



Bubble Sort algorithm





Workshop: Bubble Sort

```
#include<stdio.h>

void BubbleSort(int *b, int size, int mode)
{
    |

}

int main()
{
    int a[] = {5, 1, 4, 2, 8};
    int n = sizeof(a)/sizeof(int), mode;

    scanf("%d", &mode);    // 0 = descending order, 1 = ascending order
    BubbleSort(&a, n, mode);

    for(int i=0;i<n;i++)
        printf("%d ", a[i]);
    return 0;
}
```

ความซับซ้อนของอัลกอริทึม

- ตัวชี้วัดความซับซ้อนในการทำงานของอัลกอริทึมในเชิงเวลา (time) มีสมมติฐานว่าอัลกอริทึมจะทำงานแตกต่างกันไปตามขนาดข้อมูล (size of data) อินพุต (โดยทั่วไปใช้ตัวแปร n)
- ในการประเมินเวลาทำงานของอัลกอริทึมจะใช้วิธีนับจำนวนรอบทำงานของอัลกอริทึมเทียบกับขนาดข้อมูลอินพุต (input size) มากกว่าการจับเวลาโดยตรงจากคอมพิวเตอร์ซึ่งมักขึ้นอยู่กับฮาร์ดแวร์ (hardware) ระบบปฏิบัติการ (Operating system) และภาษาที่ใช้ในการเขียนโปรแกรม
- โดยทั่วไปการประมาณเวลาแบบนับจำนวนรอบมักจะประมาณแบบกว้างๆ (ไม่ต้องการจำนวนรอบที่แน่นอน แต่มักต้องการคำตอบเป็นจำนวนรอบสูงสุดที่เป็นไปได้) เรียกว่า **worse-case time complexity**
- **worse-case time complexity** มักเขียนในรูป $O(f(n))$ อ่านว่า big-oh ของฟังก์ชัน $f(n)$ เมื่อ $f(n)$ แทนจำนวนรอบของการทำงานซึ่งสัมพันธ์กับขนาดอินพุต n



Time complexity ของ Linear search

```
1 #include<stdio.h>
2
3 int main() {
4     int arr[] = {70,40,30,11,57,41,25,14,52};
5     int n = sizeof(arr)/sizeof(int);
6     int k = 41, i=0;
7
8     while( (arr[i] != k) && (i < n))
9         i++;
10
11     if(i<n)
12         printf("found item %d at index %d in arr", k, i);
13     else
14         printf("no %d in arr", k);
15
16     return 0;
17 }
```

อัลกอริทึม linear search จะวนลูปสัมพันธ์กับขนาดของอินพุต n (ขนาดของอาร์เรย์)

- เมื่อ n = 10, ลูป while รันสูงสุดไม่เกิน 10 รอบ
- n = 100, ลูป while รันสูงสุดไม่เกิน 100 รอบ

จะเห็นได้ว่าจำนวนรอบสูงสุดของลูป while จะไม่เกินค่าของ n ดังนั้น time complexity ของอัลกอริทึม linear search สามารถเขียนในรูปของ $O(n)$

หรือกล่าวอีกนัยว่า “การค้นหาข้อมูลใน linear search จะทำงานไม่เกิน n รอบ”

Time complexity ของ Bubble Sort

```
1  #include<stdio.h>
2
3  int main() {
4      int arr[] = {5,1,4,2,8}, temp;
5      int n = sizeof(arr)/sizeof(int);
6      for(int i=0; i<n-1; i++)
7          for(int j = i+1; j<n; j++)
8              { if(arr[i] > arr[j])
9                  { temp = arr[i];
10                     arr[i] = arr[j];
11                     arr[j] = temp;
12                 }
13             }
14      for(int i=0; i<n; i++)
15          printf("%d ", arr[i]);
16      return 0;
17 }
```

อัลกอริทึม Bubble sort มีลูปซ้อนกัน 2 ลูป ได้แก่ ลูป i ทำหน้าที่ควบคุมจำนวนการเรียงข้อมูลในแต่ละรอบ และลูป j ทำหน้าที่เปรียบเทียบสมาชิกทุกตัวที่อยู่ติดกัน

- เมื่อ $i=0$ -> ลูป j ทำงาน $n-1$ รอบ
- เมื่อ $i=1$ -> ลูป j ทำงาน $n-2$ รอบ
-
- เมื่อ $i=n-2$ -> ลูป j ทำงาน 1 รอบ

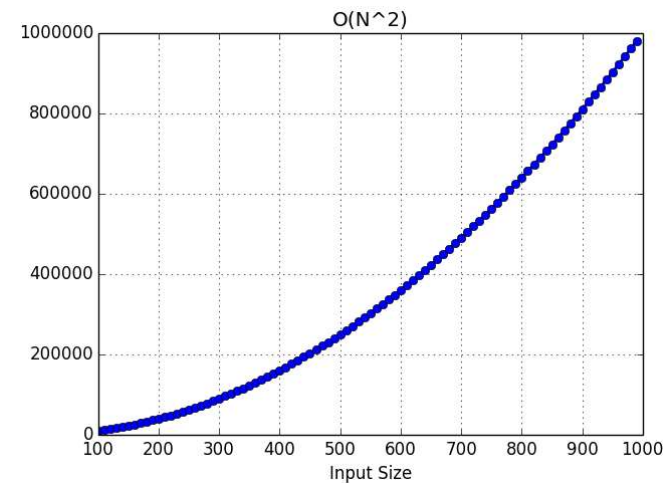
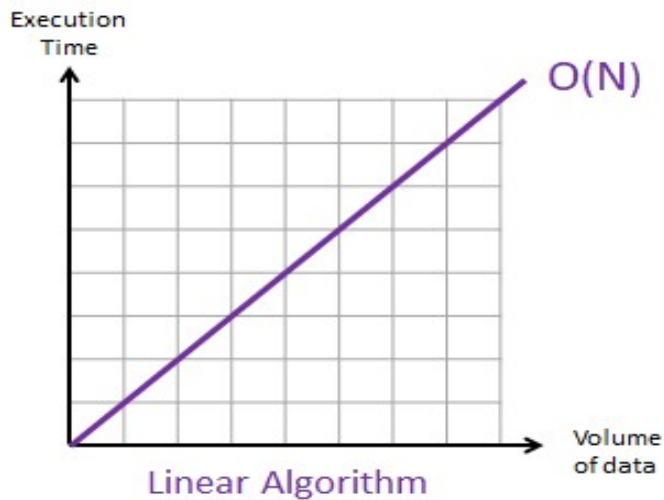
จำนวนรอบทำงานของอัลกอริทึม Bubble sort จะเกิดจากผลรวมของลูป j = $1+2+3+4+\dots+(n-2)+(n-1) \cong O(n^2)$

หรือกล่าวอีกนัยว่า “การเรียงลำดับข้อมูลใน bubble sort จะทำงานไม่เกิน n^2 รอบ”

Time Complexities

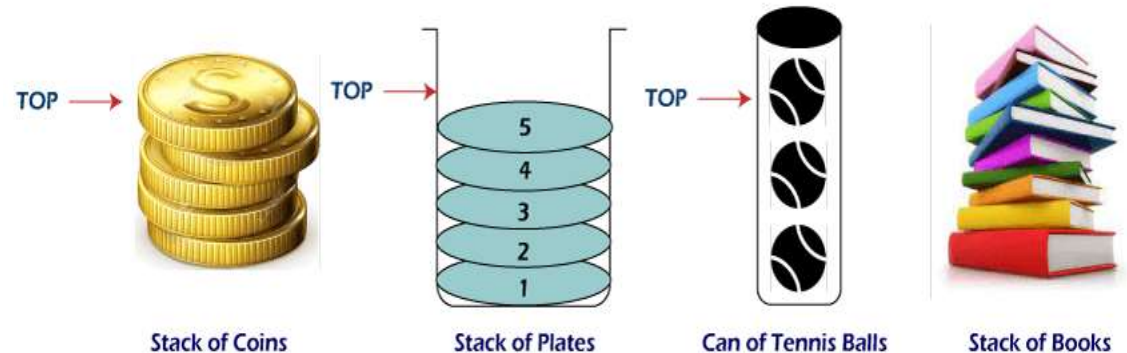
n	Linear Search
100	100
1,000	1,000
5,000	5,000
10,000	10,000
50,000	50,000
100,000	100,000

n	Bubble Sort
100	10,000
1,000	1,000,000
5000	25,000,000
10000	100,000,000
10,000	100,000,000
100,000	10,000,000,000

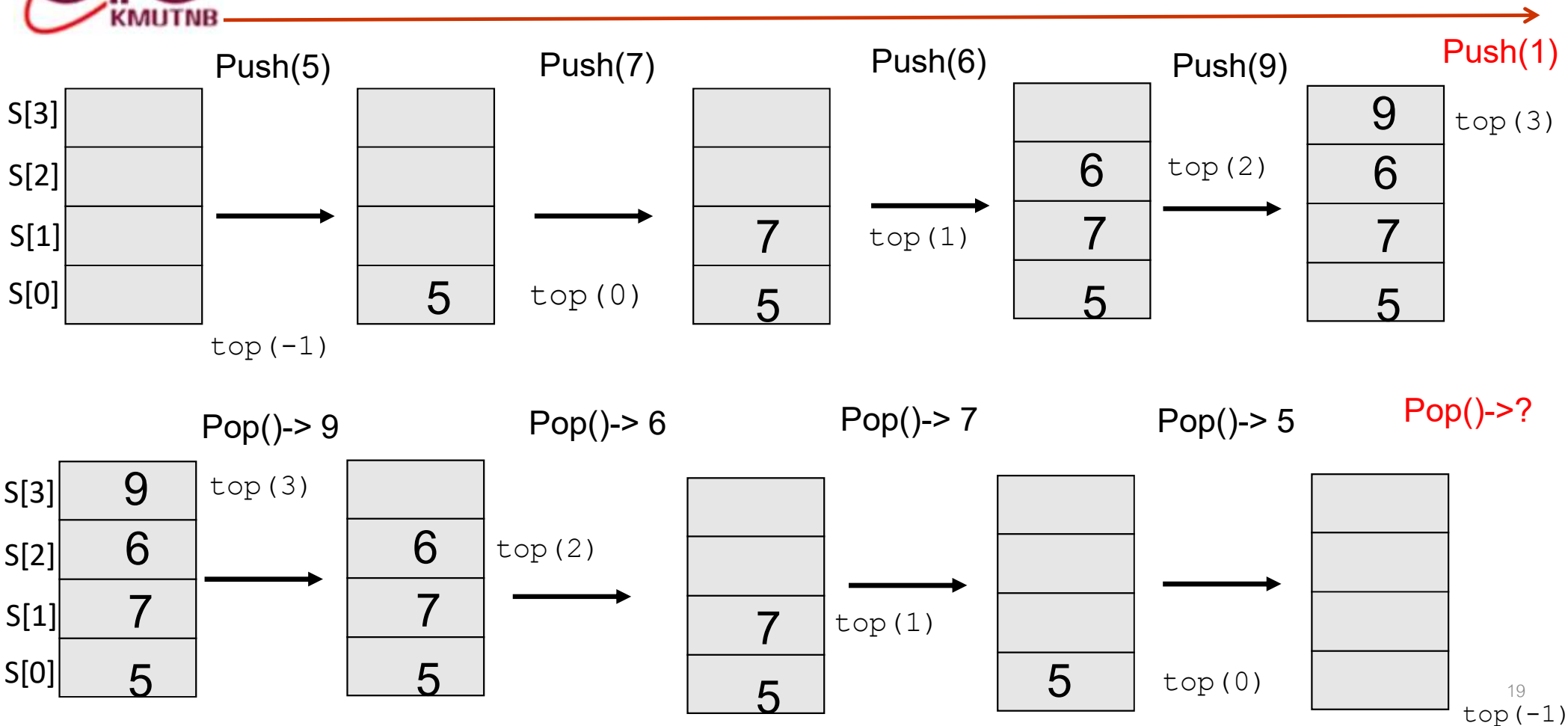


สแตก (stack)

- เป็นโครงสร้างข้อมูลเชิงเส้นประเภทหนึ่งที่สามารถพัฒนาโดยใช้อาร์เรย์
- สแตก (stack) มีการจัดการข้อมูลแบบเข้าก่อนออกทีหลัง (last in first out: LIFO) กล่าวคือ เมื่อมีการนำข้อมูลมาเก็บไว้ในสแตกจะไม่สามารถนำข้อมูลนี้ออกจากสแตกได้ จนกว่าข้อมูลที่เข้ามาทีหลังจะออกจากสแตกไปก่อนจนหมด
- สแตกเป็น abstract datatype ที่จะต้องพัฒนาโอเปอเรชั่นขึ้นใช้งานเอง ได้แก่
 - ✓ **push(x)**
 - นำเข้าข้อมูล x ไปเก็บไว้ในสแตก
 - ✓ **y = pop()**
 - นำออกข้อมูลล่าสุดที่เข้ามาในสแตก
- การควบคุมลำดับเข้าออกของข้อมูลในสแตกจะใช้ตัวแปร **top** เพื่อชี้จุดเข้าและออกในสแตก



โอเปอเรชั่น push และ pop



การพัฒนาสแตกโดยใช้อาร์เรย์

- การเตรียม Stack ด้วยอาร์เรย์

สร้างตัวแปรอาร์เรย์พร้อมกำหนดขนาดสูงสุดของสแตกที่จะรองรับข้อมูลได้

```
#define MAXSIZE 10;
```

```
int stackArray[MAXSIZE] ;
```

```
int top = -1;
```

- พัฒนาฟังก์ชัน push() และ pop() ตามหลักการทำงานของสแตก



การพัฒนาสแตกโดยใช้อาร์เรย์

```
void push(int x)
{
    if(top > MAXSIZE)
    {
        printf("STACK FULL");
    }
    else
    {
        top++;
        stackArray[top] = x;
    }
}
```

- เช็ค stack ก่อนนำ x เข้าว่าเต็ม (full) หรือไม่
- หาก stack ไม่เต็ม เพิ่มค่า top ขึ้นไป 1 แล้วนำ x ไปเก็บไว้ใน stack ในตำแหน่งปัจจุบันของ top

```
int pop()
{
    if(top == -1)
    {
        printf("STACK Empty");
        return -1;
    }
    else
    {
        y = stackArray[top];
        top--;
        return y;
    }
}
```

- เช็ค stack ว่าว่าง (empty) หรือไม่
- หาก stack ไม่ว่าง ให้นำข้อมูลออกจากตำแหน่งปัจจุบันของ top แล้วลดค่า top ลงไป 1



Example

```
push 5
push 7
push 6
push 9
9 -> 6 -> 7 -> 5
```

Time complexity of push() : $O(1)$

Time complexity of pop() : $O(1)$

```
1  #include<stdio.h>
2
3  #define MAXSIZE 10
4  int stackArray[MAXSIZE];
5  int top = -1;
6
7  void push(int x)
8  {   if(top > MAXSIZE)
9      {   printf("STACK FULL");
10         }
11      else
12      {   top++;
13          stackArray [top] = x;
14      }
15  }
16
17  int pop()
18  {   if(top == -1)
19      {   printf("STACK Empty");
20          return -1;
21      }
22      else
23      {   int y = stackArray[top];
24          top--;
25          return y;
26      }
27  }
28
29  int main()
30  {   printf("push 5\n");   push(5);
31      printf("push 7\n");   push(7);
32      printf("push 6\n");   push(6);
33      printf("push 9\n");   push(9);
34      printf("%d -> ", pop());   printf("%d -> ", pop());
35      printf("%d -> ", pop());   printf("%d", pop());
36      return 0;
```

Stack in C++ STL

- ใน C++ สแตกถูกพัฒนาสำเร็จรูปพร้อมให้ใช้งานโดยใช้ container stack

```
1 #include <iostream>
2 #include <stack>
3 using namespace std;
4 int main() {
5     stack<int> stack;
6     stack.push(5);    stack.push(7);
7     stack.push(6);    stack.push(9);
8
9     while (!stack.empty()) {
10         cout << stack.top() << " ";
11         stack.pop();
12     }
13 }
```

9 6 7 5

empty(): เช็ค stack ว่าว่าง

size(): เช็คจำนวนข้อมูล

top(): แสดงข้อมูลในตำแหน่งปัจจุบันของ top

push(x): นำเข้าข้อมูล x

pop(): นำออกข้อมูล

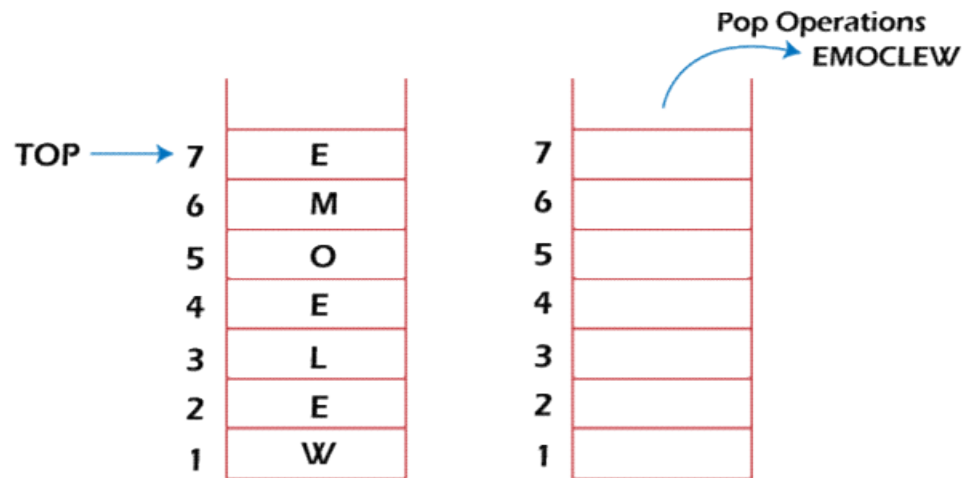


ตัวอย่างแอปพลิเคชันของสแตก

- การสร้างสตริงลำดับกลับ (string reversal)
- การเช็ควงเล็บในนิพจน์ (delimiter checking)
- การคำนวณค่าของนิพจน์ (evaluation of arithmetic expression)

สตริงลำดับกลับ (string reversal)

- เนื่องจาก stack มีการจัดลำดับข้อมูลที่เข้าและออกจาก stack แบบ LIFO
- ดังนั้นหากต้องการเรียงลำดับของข้อมูลใหม่จากหน้าไปหลัง ให้ push ข้อมูลที่ละตัวลงใน stack จนครบ และ pop ออกมา stack ว่าง
- ตัวอย่างเช่น WELEOME -> EMOCLEW



Workshop: String reversal using stack



```
#include <stdio.h>
#include <string.h>
#define MAX_SIZE 100

char stack[MAX_SIZE];

void push(char c) {

}

char pop() {

}

int main() {
    char str[MAX_SIZE];
    printf("Enter a string: ");
    gets(str);

    return 0;
}
```

จงเขียนภาษาซีเพื่อรับอินพุตสตริงจากผู้ใช้งานความยาวไม่เกิน 100
จากนั้นแสดงผลลัพธ์เป็นสตริงลำดับกลับโดยใช้สแตกตามฟังก์ชันที่
กำหนด



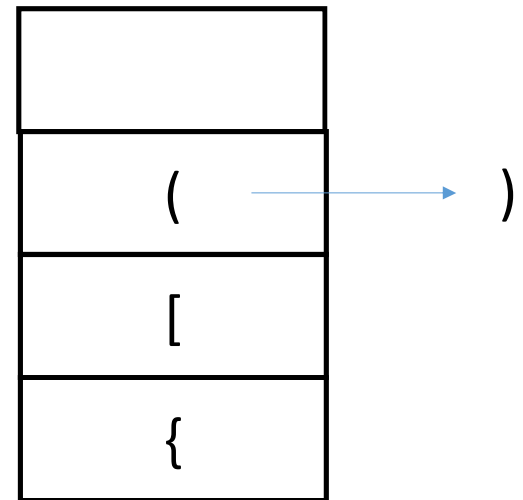
Delimiter Checking

- นิพจน์ทางคณิตศาสตร์ส่วนมากจะใช้วงเล็บ (parenthesis) เพื่อสร้างลำดับการคำนวณค่านิพจน์ ซึ่งมีหลายรูปแบบ เช่น
 - `() {} []`
- ในการตรวจสอบความถูกต้องของการใช้วงเล็บในนิพจน์จะต้องตรวจสอบ 2 ข้อ
 1. จำนวนวงเล็บเปิดและปิดจะต้องมีจำนวนเท่ากัน
 - `{ [( ) ] }` **OK**
 - `{ [( ) }` **NOT OK**
 2. วงเล็บเปิดและปิดที่จะจับคู่กันได้จะต้องเป็นประเภทเดียวกัน
 - `{ ( [ ] ) }` **NOT OK**

Delimiter checking using stack

- สามารถใช้ stack มาช่วยในการตรวจสอบความถูกต้องได้ เช่น $\{ [()] \}$

1. อ่านข้อมูลจากซ้ายไปขวาทีละตัว
2. push วงเล็บเปิดลงใน stack
3. pop วงเล็บเปิดออกจาก stack ทุกครั้งทีอ่านเจอวงเล็บปิด
4. หากวงเล็บเปิดและปิดเป็นคนละประเภท ให้จบการทำงาน
5. วนซ้ำจนกระทั่ง stack ว่าง





การคำนวณค่านิพจน์

- ในการหาผลลัพธ์ของนิพจน์ทางคณิตศาสตร์ โดยทั่วไปมักเขียนนิพจน์ในรูปของ **infix notation**
- โดยทั่วไปจะใช้ **infix expression** ในรูปของ

operand1 operator operand2

- เช่น:
 - $1 + 2$
 - $4 * 3$
- อย่างไรก็ตามหากนิพจน์มีความซับซ้อนมากขึ้น จำเป็นต้องคำนวณตามลำดับความสำคัญของโอเปอเรเตอร์:
 - $1 + 2 * 3$
 - $10 - 4 - 3$
 - $2 \wedge 3 \wedge 3$
 - $1 - 2 - 4 \wedge 5 * 3 * 6 / 7 \wedge 2 \wedge 2$
- และต้องใช้วงเล็บเป็นตัวจัดลำดับการหาผลลัพธ์ $(1-2)-((((4\wedge 5)*3)*6)/(7\wedge(2\wedge 2)))$

การคำนวณนิพจน์

- ตัวแปลภาษาสมัยใหม่จะไม่คำนวณค่าของนิพจน์แบบ infix โดยตรง เนื่องจากค่อนข้างซับซ้อน
- แต่จะแปลงนิพจน์ infix ให้อยู่ในรูปนิพจน์แบบ postfix หรือแบบ prefix ก่อนเพื่อให้คำนวณค่าได้ง่ายขึ้น
- รูปแบบของ postfix expression

operand1 operand2 operator

- เช่น: $1+2 \rightarrow 1\ 2\ +$ หรือ $4*3 \rightarrow 4\ 3\ *$
- โดยการแสดงนิพจน์ในรูปของ postfix จึงไม่จำเป็นต้องใช้วงเล็บเพื่อกำหนดลำดับความสำคัญ
 - Infix expression:
 $1 - 2 - 4 \wedge 5 * 3 * 6 / 7 \wedge 2 \wedge 2$ หรือ $(1-2)-((((4\wedge 5)*3)*6)/(7\wedge(2\wedge 2)))$
 - Postfix expression:
 $1\ 2\ -\ 4\ 5\ \wedge\ 3\ *\ 6\ *\ 7\ 2\ 2\ \wedge\ \wedge\ /\ -$



การแปลงนิพจน์ infix ไปเป็นนิพจน์ postfix

1. จากนิพจน์ infix ให้ใส่วงเล็บให้กับทุกนิพจน์ย่อยโดยการคำนึงถึงลำดับการคำนวณ (precedence order)
2. เริ่มจากนิพจน์ย่อยในสุดก่อน โดยให้ย้าย operator ไปไว้ตรงกับตำแหน่งวงเล็บปิดของนิพจน์นั้น
3. ถอดวงเล็บออกทั้งหมดก็จะได้นิพจน์แบบ postfix

เช่น $A + B * C$

$(A + (B * C))$

$(A + (B C *))$

$(A (B C *)+)$

$A B C * +$

การแปลง infix ไปเป็น postfix โดยใช้ stack

	Infix	Stack	Postfix
(a)	A+B*C-D/E		
(b)	+B*C-D/E		A
(c)	B*C-D/E	+	A
(d)	*C-D/E	+	AB
(e)	C-D/E	* +	AB
(f)	-D/E	* +	ABC
(g)	D/E	- +	ABC++
(h)	/E	-	ABC++D
(i)	E	/	ABC++D
(j)		/	ABC++DE
(k)			ABC++DE/-

1. สแกนนิพจน์จากซ้ายไปขวาทีละตัว
2. หากเป็น operand จะนำ operand ไปเรียงต่อในเอาท์พุต
3. หากเป็น operator จะนำไป push เก็บไว้ในสแตก โดยมีเงื่อนไขว่าหากในสแตกมี operator ที่สำคัญสูงกว่าหรือเท่ากันจะต้องนำออกมาจากสแตกทั้งหมดก่อน จึงจะ push operator นั้นเข้าไปใน stack ได้

การคำนวณนิพจน์

Infix expression	Postfix expression
5 * 6 - 10	5 6 * 10 -
((5 * 6) - 10)	30 10 -
(30 - 10)	20
20	

การหาผลลัพธ์นิพจน์ postfix โดยใช้ stack

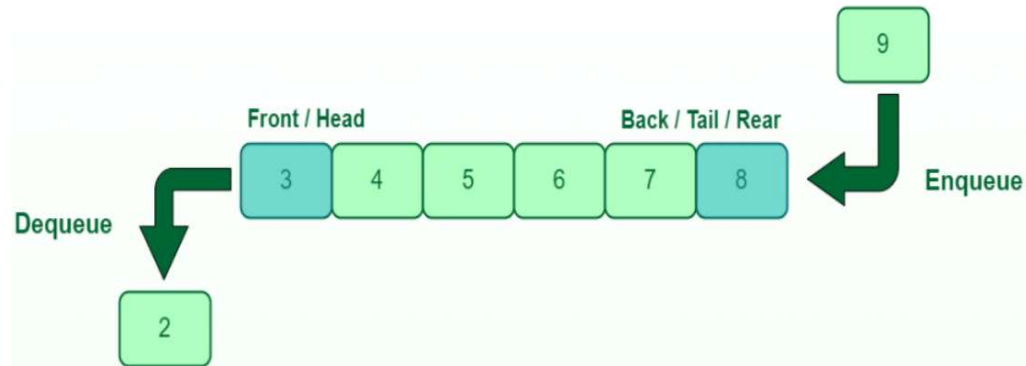
	Postfix	Stack	
(a)	2 4 6 + *		1. สแกนนิพจน์ postfix จากซ้ายไปขวาทีละตัว
(b)	4 6 + *	2	2. หากเป็น operand จะนำไปใส่ในสแตก
(c)	6 + *	4 2	3. หากเป็น operator จะทำ pop operator จากสแตก 2 ครั้ง ติดต่อกัน เพื่อนำออกมาคำนวณกับ operator ผลลัพธ์ของการ คำนวณจะถูก push ลงในสแตกอีกครั้ง
(d)	+ *	6 4 2	4. ทำแบบนี้ไปเรื่อยๆ จนกระทั่งเหลือสมาชิกในสแตกเพียงตัวเดียว
(e)	*	10 2	
(f)		20	

$4 + 6 = 10$
 $2 * 10 = 20$



คิว (Queue)

- คิว (queue) เป็นโครงสร้างข้อมูลเชิงเส้นที่มีลำดับการนำเข้าและส่งออกข้อมูลที่แตกต่างจาก stack กล่าวคือ คิวจะมีลำดับการจัดการข้อมูลแบบเข้าก่อนออกก่อน (First in First out: FIFO)
- โอเปอเรชันของคิว
 - `enqueue(x)` นำข้อมูล x ไปเก็บไว้ต่อท้ายคิว
 - `dequeue()` นำข้อมูลออกจากหน้าคิว
- การจัดการข้อมูลในคิวจะใช้ตัวแปร 2 ตัว ได้แก่
 - `front / head` : ชี้ตำแหน่งที่จะนำข้อมูลออกจากหน้าคิว
 - `rear / tail` : ชี้ตำแหน่งที่จะนำข้อมูลมาเก็บไว้ในคิว

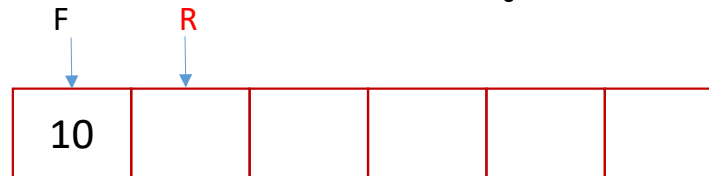


คิวแบบง่าย (Simple queue)

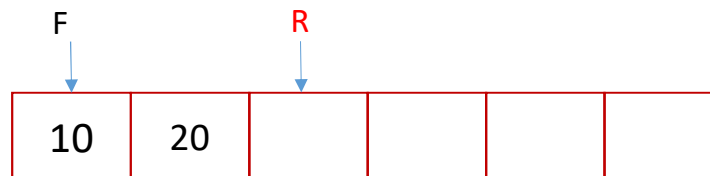
- ต้องมีการกำหนดขนาดของคิว (queue size) ให้เพียงพอกับข้อมูลที่จะนำเข้ามา
- เมื่อต้องการนำข้อมูลเข้าคิว (enqueue) จะเพิ่มข้อมูลลงในตำแหน่ง rear จากนั้นเพิ่มค่า rear ขึ้นไป 1
- เมื่อต้องการนำข้อมูลออกจากคิว (dequeue) จะนำข้อมูลในตำแหน่ง front จากนั้นเพิ่มค่า front ขึ้นไป

1

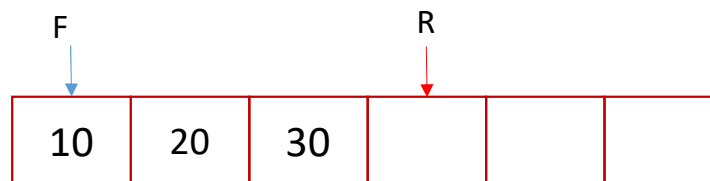
enqueue(10)



enqueue(20)

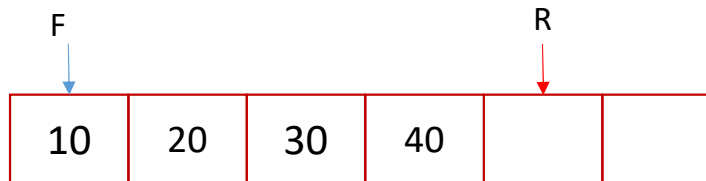


enqueue(30)

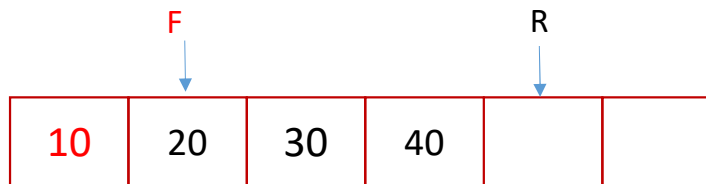


คิวแบบง่าย (Simple queue)

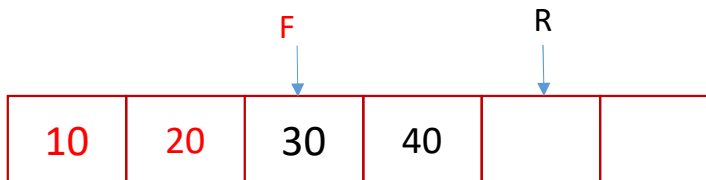
enqueue(40)



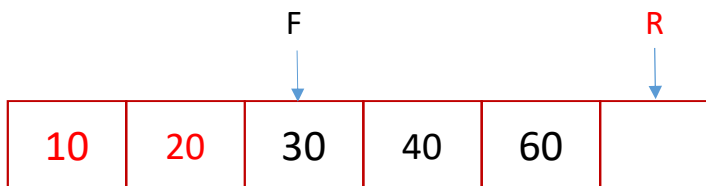
dequeue()



dequeue()



enqueue(60)



เมื่อ **rear** ใส่ข้อมูลไปจนถึงคิวตำแหน่งสุดท้าย
ก็จะไม่สามารถเพิ่มได้อีก เรียกว่า คิวเต็ม



คิวแบบง่าย (Simple queue)

```
1  #include <stdio.h>
2
3  #define MAX_SIZE 100
4  int queue[MAX_SIZE];
5  int front = -1;
6  int rear = -1;
```

```
void enqueue(int x) {
    if (is_full()) {
        printf("Error: Queue is full\n");
        return;
    } else if (is_empty()) {
        front = rear = 0;
    } else {
        rear++;
    }
    queue[rear] = x;
}
```

```
int is_empty() {
    return (front == -1 && rear == -1);
}
```

```
int is_full() {
    return (rear == MAX_SIZE - 1);
}
```

```
int dequeue() {
    if (is_empty()) {
        printf("Error: Queue is empty\n");
        return;
    } else if (front == rear) {
        front = rear = -1;
    } else {
        return queue[front++];
    }
}
```



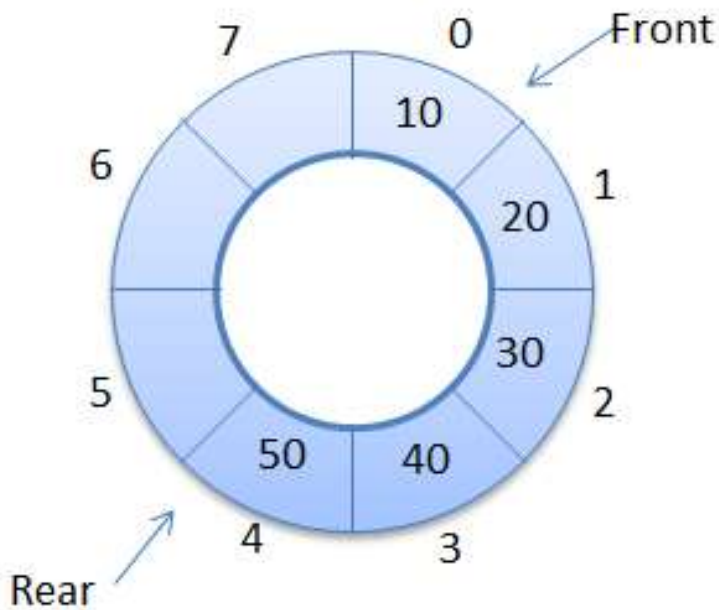
Simple Queue in C++ STL

```
The queue Q is :    10  20  30
gquiz.size() : 3
gquiz.front() : 10
gquiz.back() : 30
gquiz.pop() :    20  30
```

```
1  #include <iostream>
2  #include <queue>
3
4  using namespace std;
5
6  void showq(queue<int> gq)
7  {   queue<int> g = gq;
8      while (!g.empty()) {
9          cout << '\t' << g.front();
10         g.pop();
11     }
12     cout << '\n';
13 }
14
15 int main()
16 {   queue<int> Q;
17     Q.push(10);   Q.push(20);   Q.push(30);
18
19     cout << "The queue Q is : ";
20     showq(Q);
21
22     cout << "\ngquiz.size() : " << Q.size();
23     cout << "\ngquiz.front() : " << Q.front();
24     cout << "\ngquiz.back() : " << Q.back();
25
26     cout << "\ngquiz.pop() : ";
27     Q.pop();
28     showq(Q);
29
30     return 0;
31 }
```

คิวงกลม (Circular queue)

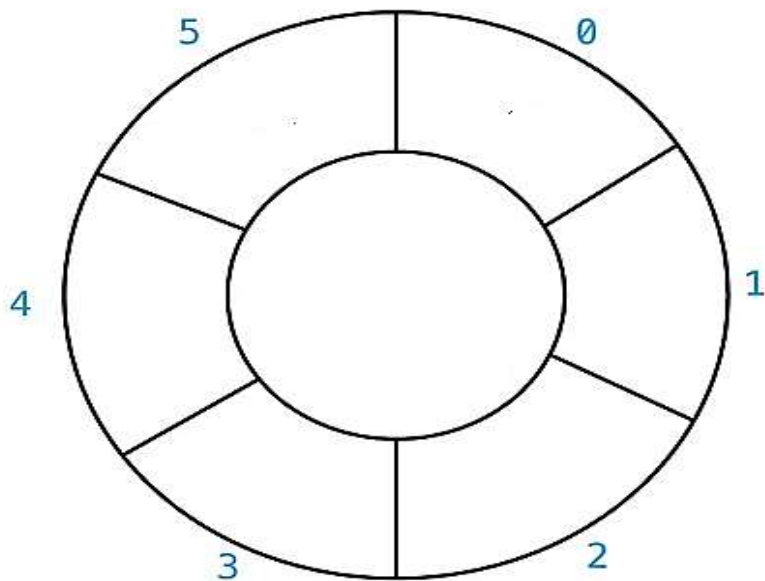
- คิวงกลมถูกออกแบบมาเพื่อแก้ไขข้อจำกัดของคิวแบบง่ายเมื่อ Rear ถูกเพิ่มค่าไปจนถึงตำแหน่งสุดท้ายของคิวก็ไม่สามารถนำข้อมูลต่อไปได้ถึงแม้ว่าคิวจะยังมีตำแหน่งว่าง (หน้า front) อยู่ก็ตาม



- คิวในตำแหน่งแรก (0) และตำแหน่งสุดท้าย (7) จะถูกยึดโยงเข้าด้วยกัน เพื่อที่ว่า Rear จะสามารถวนกลับมาที่ตำแหน่งแรกได้หากพบว่างอยู่
- ข้อดีของ circular queue คือสามารถใช้พื้นที่ในคิวได้อย่างมีประสิทธิภาพ

แบบฝึกหัด

- กำหนดให้คิววงกลมมีขนาด 6 ตำแหน่ง (0 – 5) จงแสดงผลลัพธ์ในคิว เมื่อมีการเรียกโอเปอเรชั่นตามลำดับ



1. enqueue(5)
2. enqueue(10)
3. enqueue(15)
4. enqueue(20)
5. enqueue(25)
6. enqueue(30)
7. dequeue()
8. dequeue()
9. enqueue(35)
10. enqueue(40)
11. dequeue()



Workshop 3: Circular Queue

```
#include <stdio.h>
#define SIZE 6
```

```
int front = -1;
int rear = -1;
int queue[SIZE];
```

```
int isEmpty() {
```

```
}
```

```
int isFull() {
```

```
}
```

```
void enqueue(int value) {
```

```
}
```

```
int dequeue() {
```

```
}
```

```
void display() {
```

```
    printf("Queue elements are:\n");
```

```
    for (int i = 0; i < SIZE; i++) {
```

```
        printf("%d ", queue[i]);
```

```
    }
```

```
    printf("\n");
```

```
}
```

```
int main() {
    enqueue(5); display();
    enqueue(10); display();
    enqueue(15); display();
    enqueue(20); display();
    enqueue(25); display();
    enqueue(30); display();
```

```
    dequeue(); display();
```

```
    dequeue(); display();
```

```
    enqueue(25); display();
```

```
    enqueue(30); display();
```

```
    dequeue(); display();
```

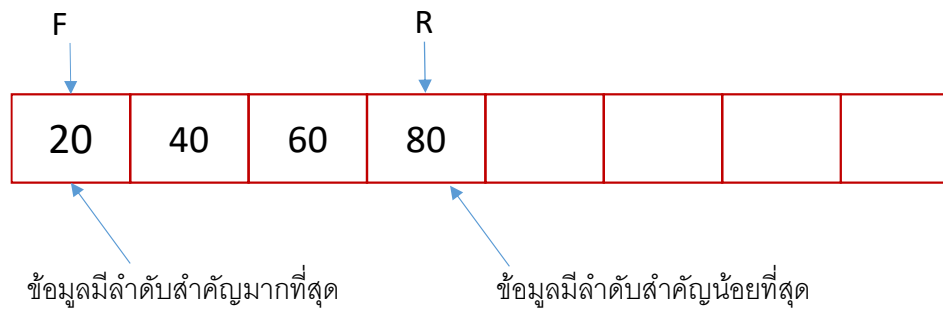
```
    return 0;
```

```
}
```

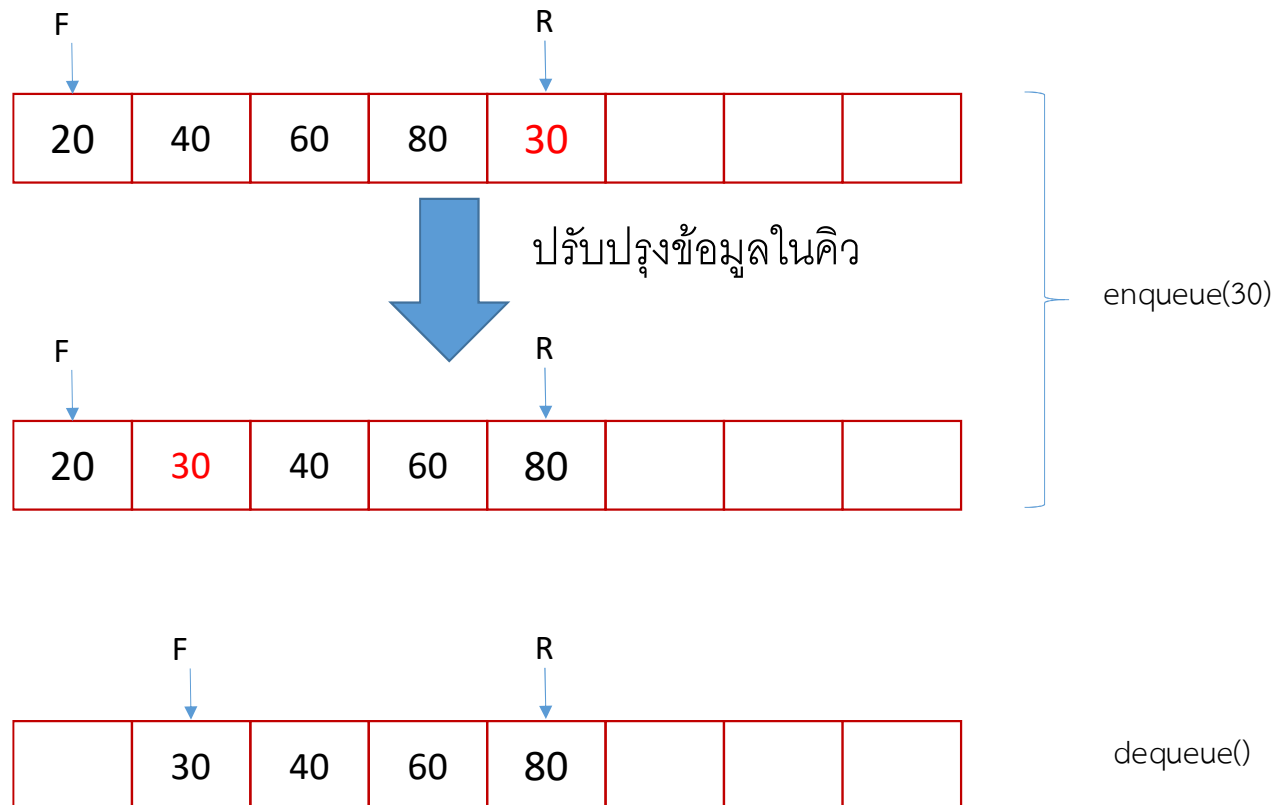


คิวลำดับความสำคัญ (Priority Queue)

- จัดเก็บข้อมูลตามลำดับความสำคัญ (Priority) จากมากไปน้อย หรือน้อยไปมากก็ได้
- ข้อมูลที่มีลำดับสำคัญที่สุดจะอยู่ที่หน้าคิว (Front) และข้อมูลลำดับสำคัญน้อยที่สุดจะอยู่ท้ายคิว (Rear)
- ทุกครั้งที่มีการเพิ่มข้อมูลใหม่ จะต้องมีการปรับปรุงข้อมูลในคิว



คิวลำดับความสำคัญ (Priority queue)





Priority Queue in C++ STL

- รูปแบบ

`std::priority_queue<int> pq;`

Array: 10 2 4 8 6 9

Priority Queue: 10 9 8 6 4 2

```
1  #include <iostream>
2  #include <queue>
3  using namespace std;
4
5  int main()
6  {
7      int arr[6] = { 10, 2, 4, 8, 6, 9 };
8
9      priority_queue<int> pq; // max priority queue
10     // priority_queue <int, vector<int>, greater<int>> pq; // min priority queue
11
12     cout << "Array: ";
13     for (auto i : arr) {
14         cout << i << ' ';
15     }
16     cout << endl;
17     for (int i = 0; i < 6; i++) {
18         pq.push(arr[i]);
19     }
20
21     cout << "Priority Queue: ";
22     while (!pq.empty()) {
23         cout << pq.top() << ' ';
24         pq.pop();
25     }
26 }
```



Workshop 4: Kth Maximum value

- จงเขียนโปรแกรมเพื่อรับอินพุตเป็นอาร์เรย์จำนวนเต็ม n จำนวน จากนั้นค้นหาสมาชิกที่มีค่ามากที่สุดลำดับที่ K ในอาร์เรย์ โดยไม่ต้องอนุญาตให้เรียงข้อมูลก่อน ตัวอย่างเช่น

$a[] = \{3, 5, 2, 6, 8, 9, 1\}$; และ $k = 3$

สมาชิกที่มีค่ามากที่สุดลำดับ 3 คือ 6

```
#include <iostream>
#include <queue>

using namespace std;

int K_Max(int *nums, int n, int k) {

}

int main() {
    int nums[] = {3, 5, 2, 6, 8, 9, 1};
    int n = sizeof(nums)/sizeof(int);
    int k = 3;
    cout << "The third maximum element is: " << K_Max(nums, n, k) << endl;

    return 0;
}
```

Time complexity in queue

	Simple Queue	Circular Queue	Priority Queue
enqueue(x)	$O(1)$	$O(1)$	$O(n)$
dequeue()	$O(1)$	$O(1)$	$O(1)$